

Deployment of 3-Tier Application Using AWS Services

A Traditional Cloud Deployment for a Web Application with Database Backend

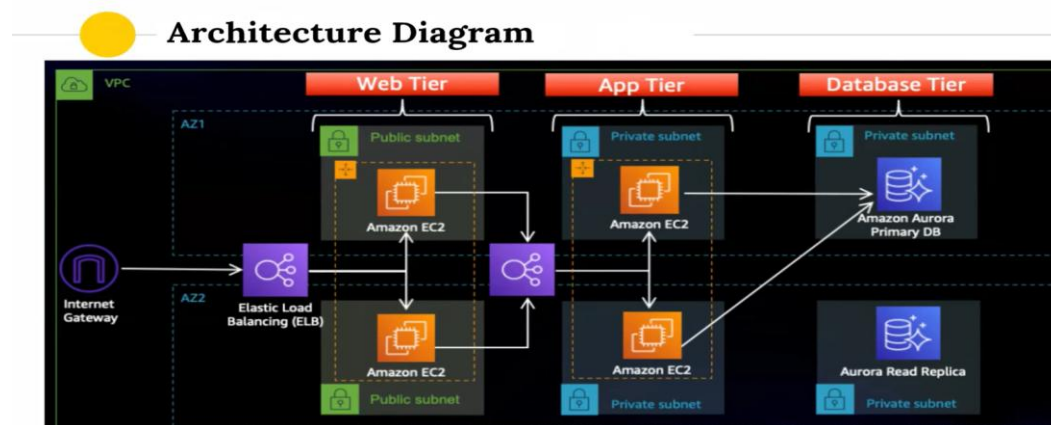
Domain: Cloud Computing and DevOps

Project Overview: This project demonstrates a manually deployed 3-tier architecture on AWS for a web application where users can submit data, which is then stored in a backend database. The setup does not use DevOps tools (like Jenkins, Docker, or Kubernetes) and relies on AWS native services for hosting, networking, and storage.

Key Components:

- Presentation Tier (Frontend) – Hosted on AWS S3 (Static Website).
- Application Tier (Backend) – Runs on AWS EC2 (Web Server).
- Database Tier – Managed by AWS RDS (MySQL/PostgreSQL).
- Networking & Security – VPC, IAM, Route 53 for access control and DNS.

What is 3 Tier Architecture?



3-Tier Architecture is a software design pattern that divides an application into **three separate layers (tiers)**. Each tier has a specific responsibility, which makes the system **more scalable, secure, and easy to maintain**.

1. Presentation Tier (UI Layer)
2. Application Tier (Business Logic Layer)
3. Data Tier (Database Layer)

3 Tier Architecture separates concerns, making applications robust, flexible, and easier to develop and manage.

So, Let's Start The Project

Step 1: Launching The AWS Console

Design and create a Virtual Private Cloud (VPC) to serve as the foundation for the project infrastructure.

- Setting up VPC and Subnets

The screenshot displays the AWS Management Console for a Virtual Private Cloud (VPC) in the Asia Pacific (Mumbai) region. The console is titled "VPC" and shows the "Your VPCs" section. The "demo-vpc" is selected, and the "Resource map" tab is active. The resource map shows the VPC structure, including subnets and route tables.

VPC
Your AWS virtual network

Subnets (6)
Subnets within this VPC

- ap-south-1a**
 - demo-vpc-subnet-public1-ap-south-1a
 - demo-vpc-subnet-db1-ap-south-1a
 - demo-vpc-subnet-app1-ap-south-1a
- ap-south-1b**
 - demo-vpc-subnet-public2-ap-south-1b
 - demo-vpc-subnet-app2-ap-south-1b
 - demo-vpc-subnet-db2-ap-south-1b

Route tables (6)
Route network traffic to resources

- demo-vpc-rtb-private3-ap-south-1a
- demo-vpc-rtb-private1-ap-south-1a
- demo-vpc-rtb-public
- rtb-049da34c4f3034438
- demo-vpc-rtb-private4-ap-south-1b
- demo-vpc-rtb-private2-ap-south-1b

The console also shows a sidebar with navigation options for VPC, Security, and PrivateLink and Lattice. The bottom status bar indicates the current time is 2:29 PM on 28-Dec-25.

- Configuring Security Group and Inbound Rules.

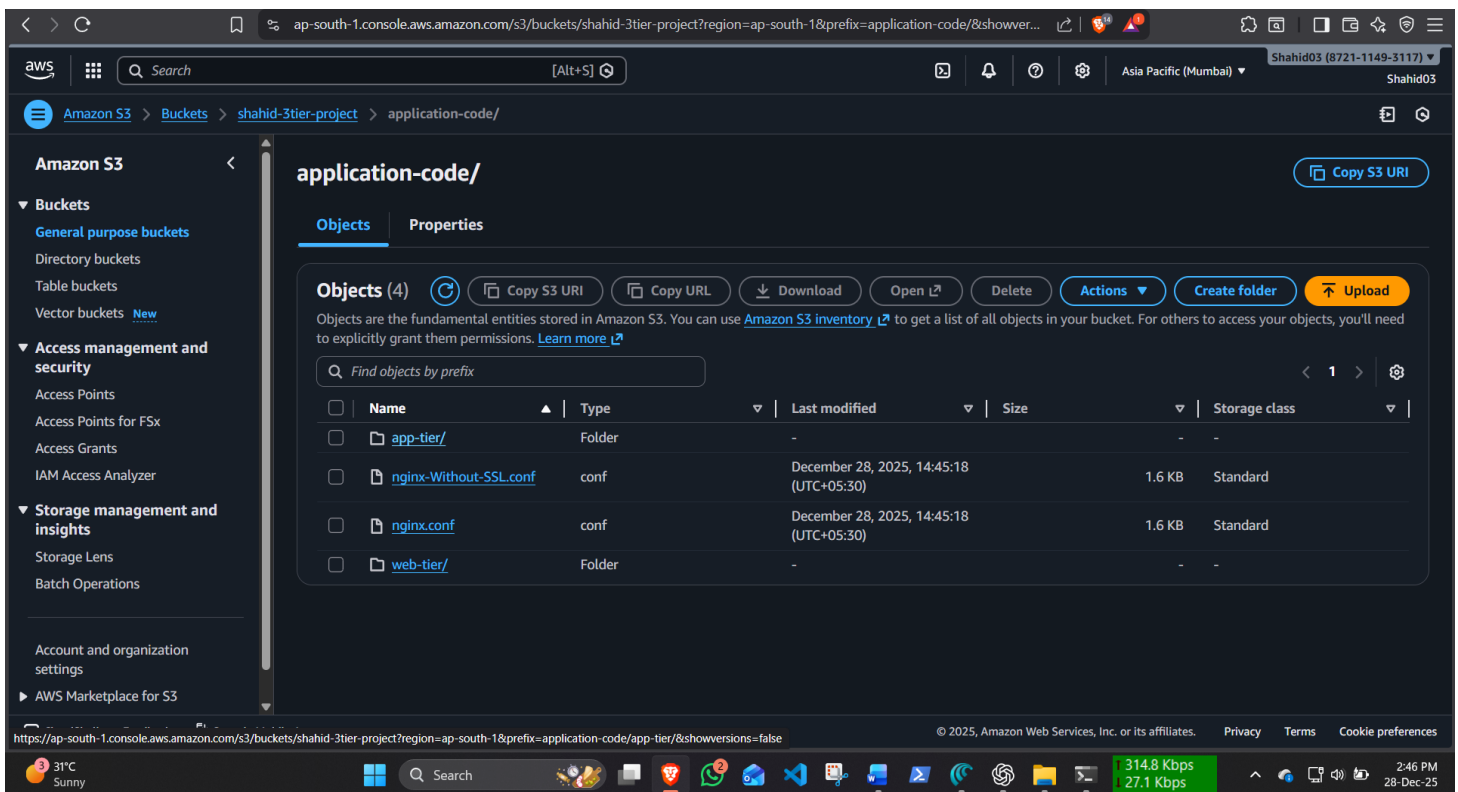
The screenshot shows the AWS Management Console for the 'ap-south-1' region. The 'Security Groups' page is active, displaying a list of 7 security groups. The table below summarizes the groups shown:

Name	Security group ID	Security group name	VPC ID	Description
-	sg-02feac3308ac1185d	default	vpc-00908c66dd21a315e	default VPC securit
-	sg-02173919ff60516ff	default	vpc-0af610dd19cf61325	default VPC securit
-	sg-0a2dec35fb9f8e9a	Web-SG	vpc-0af610dd19cf61325	Web-SG
-	sg-08b903418abe26066	App-SG	vpc-0af610dd19cf61325	App-SG
-	sg-02993829a9ad7db64	RDS-SG	vpc-0af610dd19cf61325	RDS-SG
-	sg-0e887f067a8bf050b	Web-ALB-SG	vpc-0af610dd19cf61325	Web-ALB-SG
-	sg-015175fa8b187a49c	internal-ALB-SG	vpc-0af610dd19cf61325	internal-ALB-SG

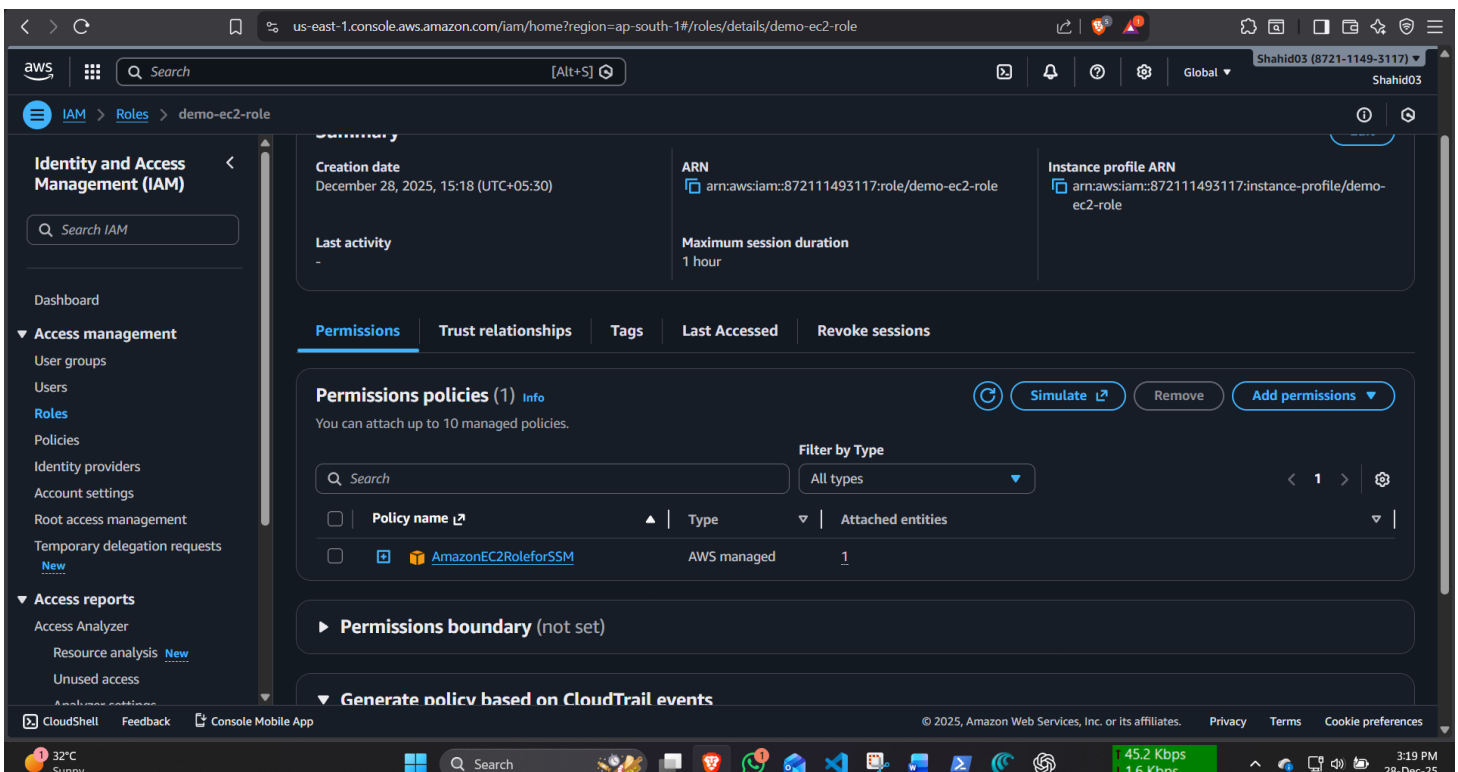
The console interface includes a sidebar with navigation links for VPC, Security, and PrivateLink and Lattice. The bottom status bar shows the system is 'Sunny' at 31°C, with a network speed of 53.3 Kbps. The footer includes copyright information for Amazon Web Services, Inc. (© 2025) and links to Privacy, Terms, and Cookie preferences.

Step 2: S3 Bucket and IAM Role Setup

- Create an S3 bucket and upload the application code.

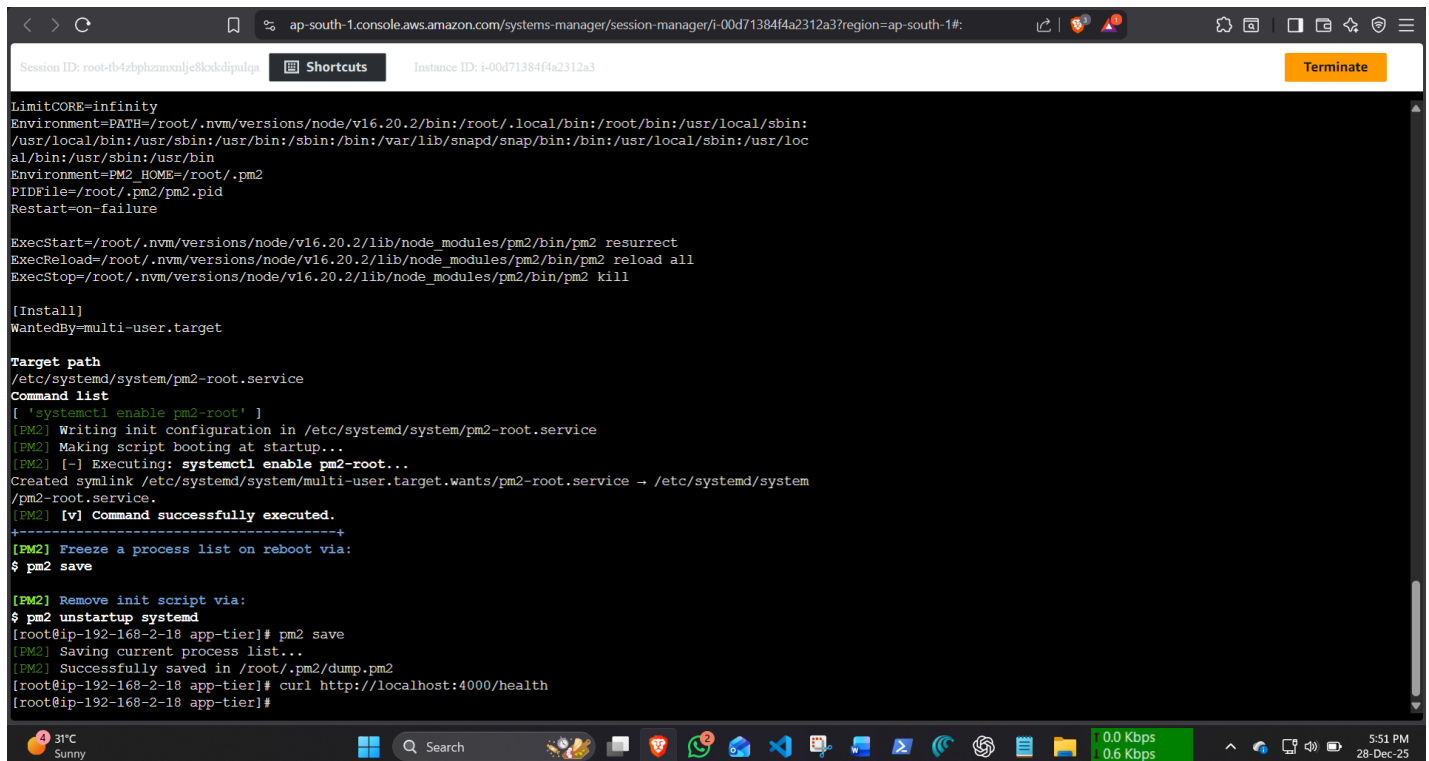


- Set up an IAM role with the necessary permissions and attach it to the EC2 instance.



Step3: Creation of App Tier Resources

- Database Configuration
- Launch and configure an RDS instance to serve as the backend database.
- Creating database
- Creating tables and adding information
- Creating internal load balancer
- Configuring nginx.config file
- Uploading the data to s3 bucket
- Configuring all the necessary dependencies to run our application
- Installing nginx and configuring it



The screenshot displays an AWS Systems Manager session interface. At the top, the browser address bar shows the URL: `ap-south-1.console.aws.amazon.com/systems-manager/session-manager/i-00d71384f4a2312a3?region=ap-south-14:`. Below the address bar, the session ID is `root-tb4zphzmxn1je8kxkddipulga` and the instance ID is `i-00d71384f4a2312a3`. A "Terminate" button is visible in the top right corner. The main terminal window shows the following commands and output:

```
LimitCORE=infinity
Environment=PATH=/root/.nvm/versions/node/v16.20.2/bin:/root/.local/bin:/root/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/var/lib/napd/snap/bin:/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
Environment=PM2_HOME=/root/.pm2
PIDFile=/root/.pm2/pm2.pid
Restart=on-failure

ExecStart=/root/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 resurrect
ExecReload=/root/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 reload all
ExecStop=/root/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 kill

[Install]
WantedBy=multi-user.target

Target path
/etc/systemd/system/pm2-root.service
Command list
[ 'systemctl enable pm2-root' ]
[PM2] Writing init configuration in /etc/systemd/system/pm2-root.service
[PM2] Making script booting at startup...
[PM2] [-] Executing: systemctl enable pm2-root...
Created symlink /etc/systemd/system/multi-user.target.wants/pm2-root.service -> /etc/systemd/system/pm2-root.service.
[PM2] [v] Command successfully executed.
-----+
[PM2] Freeze a process list on reboot via:
$ pm2 save

[PM2] Remove init script via:
$ pm2 unstartup systemd
[root@ip-192-168-2-18 app-tier]# pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /root/.pm2/dump.pm2
[root@ip-192-168-2-18 app-tier]# curl http://localhost:4000/health
[root@ip-192-168-2-18 app-tier]#
```

The bottom of the screenshot shows a Windows taskbar with a search bar, several application icons, and a system tray displaying the temperature (31°C), network status (0.0 Kbps / 0.6 Kbps), and the time (5:51 PM, 28-Dec-25).

Step 4:

Creation of Web tier resources including External Load Balancer

```
sudo -su ec2-user (To work as an ec2-user)
```

```
cd /home/ec2-user
```

```
curl -o- https://raw.githubusercontent.com/avizway1/aws_3tier_architecture/main/install.sh | bash
```

```
source ~/.bashrc
```

```
nvm install 16
```

```
nvm use 16
```

```
aws s3 cp s3://<S3 Bucker Name>/application-code/web-tier/ web-tier --recursive
```

```
ls ----> You will see 'web-tier'
```

```
cd web-tier
```

```
npm install
```

```
npm run build
```

```
sudo amazon-linux-extras install nginx1 -y
```

Update Nginx configuration:

```
cd /etc/nginx (Your are in nginx path)
```

```
ls ----> You will see 'nginx.conf' file
```

```
sudo rm nginx.conf
```

```
sudo aws s3 cp s3://<S3 Bucker Name>/application-code/nginx.conf .
```

```
sudo service nginx restart
```

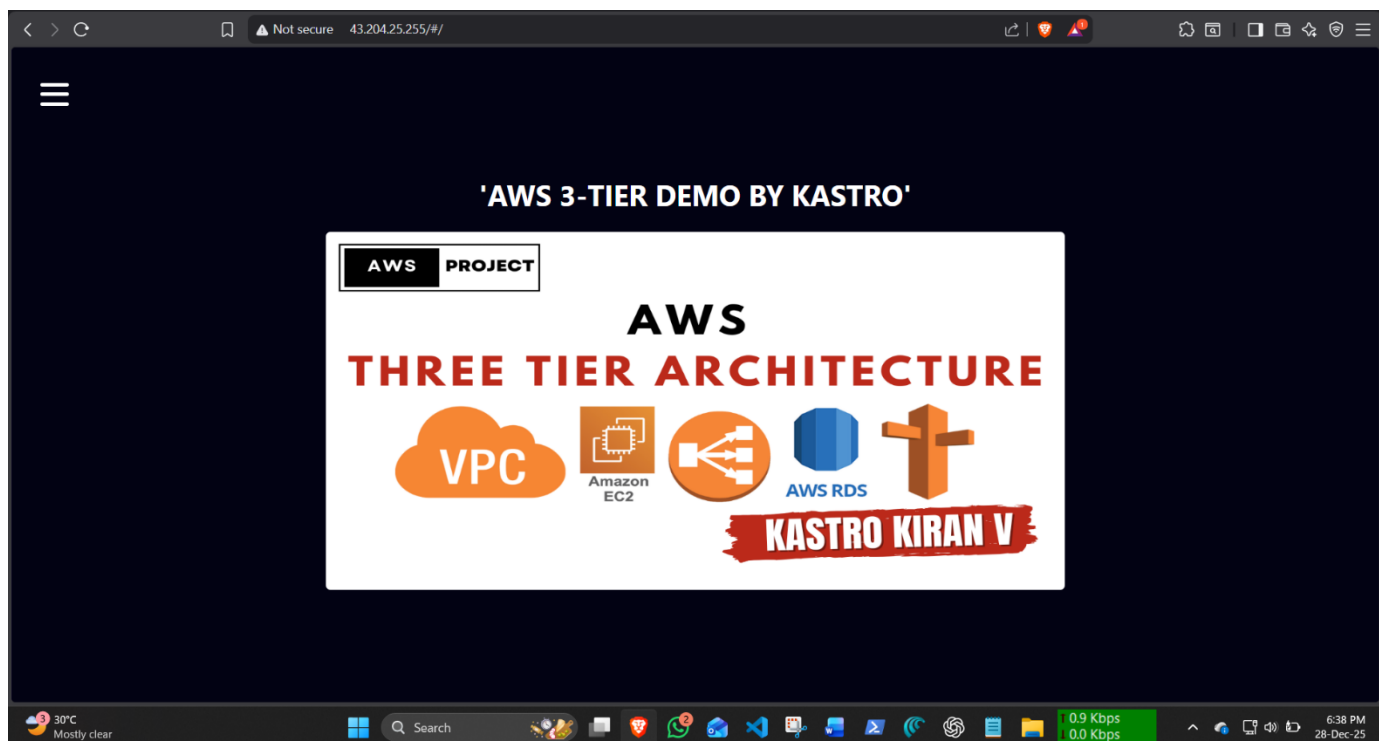
```
chmod -R 755 /home/ec2-user
```

```
sudo chkconfig nginx on
```

```
Session ID: root-d2laqlf77colum9sevr47layu Instance ID: i-07d822addf1af7969 Terminate
```

```
ss': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/DatabaseDemo/DatabaseDemo.js': Operation not permitted
s': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/Home': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/Home/Home.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/Menu': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/Menu/Menu.styled.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/Menu/index.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/Menu/Menu.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/components/index.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/App.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/global.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/index.css': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/hooks.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/App.css': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/theme.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/setupTests.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/App.test.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/index.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/src/reportWebVitals.js': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/package.json': Operation not permitted
chmod: changing permissions of '/home/ec2-user/app-tier/README.md': Operation not permitted
[ec2-user@ip-192-168-0-16 nginx]$ sudo chkconfig nginx on
Note: Forwarding request to 'systemctl enable nginx.service'.
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service -> /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-192-168-0-16 nginx]$
```

Step 6: Accessing Our App Using Public IP i.e. (43.204.25.255)



Step 7: Using Amazon Route53

Accessing The Web-App using my custom Domain (www.dodevops.ninja)



Conclusion

This project successfully demonstrated the manual deployment of a traditional 3-tier web application architecture on AWS using native cloud services.

By separating the presentation, application, and database tiers across S3, EC2, and RDS respectively, the solution achieved clear isolation, scalability, and security. The use of VPC networking, private subnets, security groups, and IAM roles ensured that access was tightly controlled and aligned with AWS security best practices. Integrating Route 53 enabled seamless access through a custom domain, enhancing usability and professionalism. Overall, this project provided strong hands-on experience with core AWS services and validated how a reliable, secure, and production-ready web application can be built without relying on DevOps automation tools, reinforcing fundamental cloud architecture concepts.